拆解TileLang/Tilekernels开源库
——TileLang原生支持昇腾950背后的技术

作者：王凯、孙文博、张雨晨

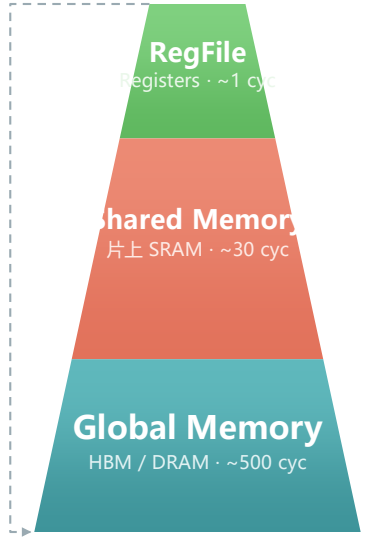
时间：2026/10/12

CANN

TileLang编程范式

1 内存层级与计算模型

越往上容量越小、延迟越低：每一级只保留当下要用的 tile



T.gemm 计算

T.copy Shared → RegFile

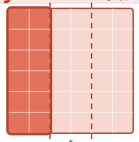
T.copy Global → Shared

RegFile • 元素分给线程，成 fragment



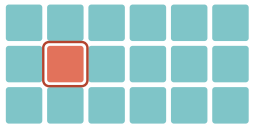
T.copy

Shared Memory • block 装入 tile，沿 K 再切



T.copy

Global Memory • 大矩阵切成 block



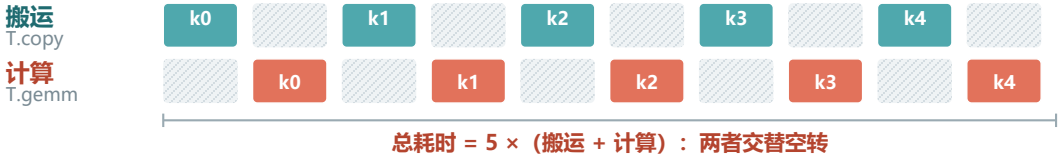
虚线：结果回写 T.copy(C_local, C_global)

同一份数据逐级缩小：大矩阵 → block → sub-tile → fragment

2 串行流水 • T.Serial

每轮先搬后算：搬运时计算单元空闲，计算时搬运引擎空闲

T.copy 搬运　T.gemm 计算



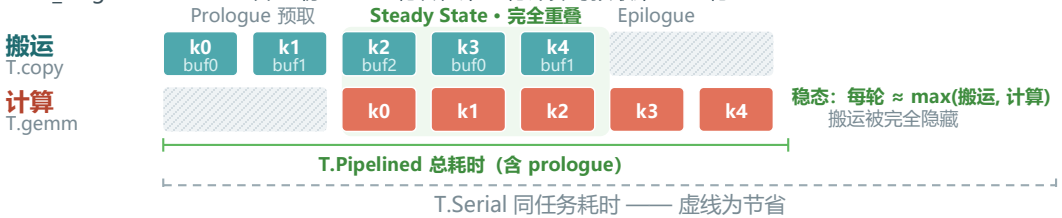
总耗时 = 5 ×（搬运 + 计算）：两者交替空转

```
for k in T.serial(T.ceildiv(K, block_K)):
```

计算单元利用率 ≈ 50%，搬运引擎同样一半时间空转

3 多 Buffer 流水 • T.Pipelined

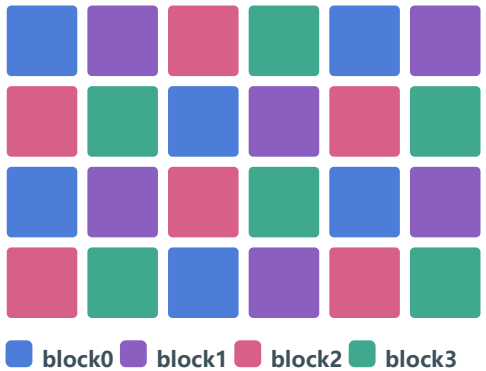
num_stages=3：Shared 开 3 份 buffer 轮转，第 k 轮计算时预取第 k+2 轮




```
for k in T.Pipelined(T.ceildiv(K, block_K), num_stages=3):
```

4 多 block 切分 • T.Persistent

常驻 block 数固定，每个 block 循环扫完剩余的 tile




```
num_blocks = driver.get_num_sms()
with T.Kernel(num_blocks, threads=256) as block_id:
  for bx, by in T.Persistent(
      [grid_m, grid_n], num_blocks, block_id):
    ... # 本 block 处理 grid_m*grid_n/num_blocks 个 tile
```

- 非 Persistent：一个 tile 一个 block，由硬件 wave 调度，尾波常不满
- T.Persistent：block 数固定，block 内 for 循环扫描剩余 tile
- 效果：launch 开销小、L2 复用更好、负载均衡稳定

24 个 tile ÷ 4 block = 每 block 6 个（strided）

5 多线程并行 • T.Parallel

把 tile 内的元素循环展平后按线程切分，每线程持有一部分元素并自动向量化



虚线框 = thread 2 持有的 8 个元素

```
for i, j in T.Parallel(block_M, block_N):
    C_local[i, j] = T.max(C_local[i, j], 0)
# 等价于 thread = (i*block_N + j) % threads
# 索引以 num_threads 为步长推进，自动向量化
```

- 一个 T.Parallel 循环 = 全线程协作的 element-wise 操作
- LayoutInference 自动推导 fragment 的线程/寄存器布局
- 常见于 epilogue：激活、缩放、类型转换、写回 Global

TileLang Ascend 950内存层级

1 Ascend 950 存储层级

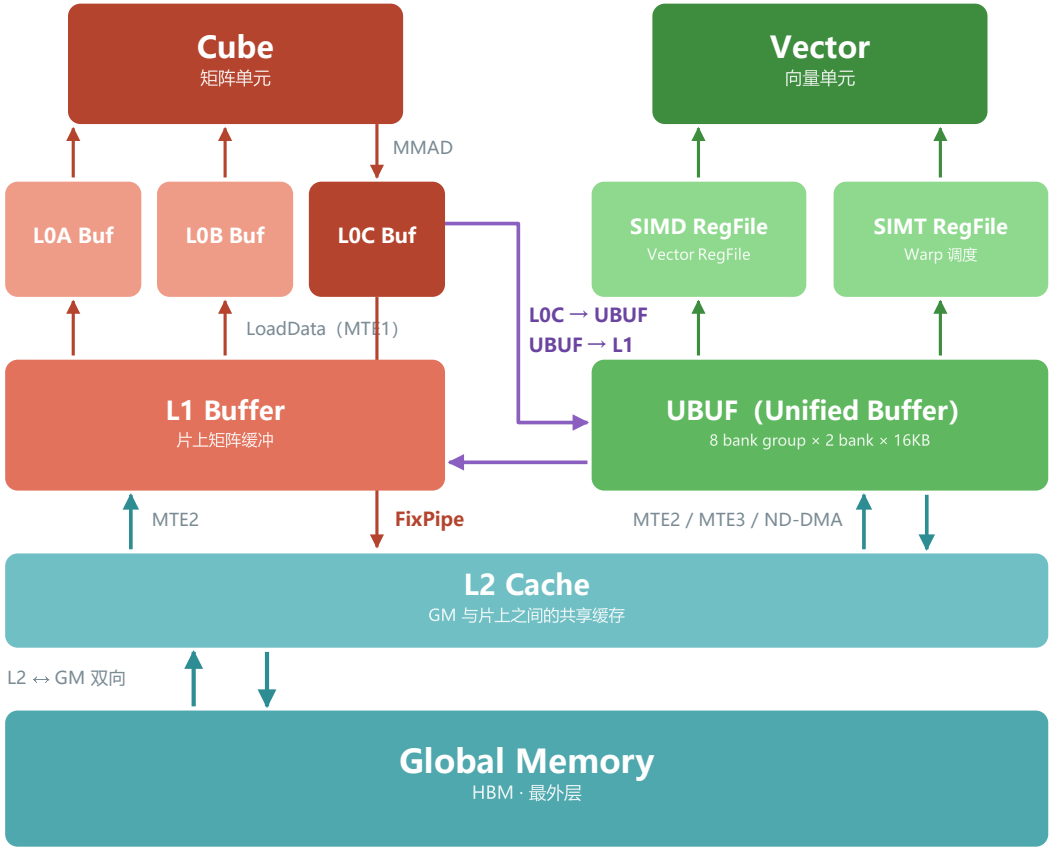
Cube / Vector 双流水；L0A/L0B/L0C 同级

CUBE / AIC • 矩阵通路

Cube 的累加写 L0C；L0A/L0B 不再直连 GM，必须由 L1 供给

VECTOR / AIV • 向量通路

Regbase 架构：AIV 直接操作 Vector 寄存器，结果写回 UBUF



2201 → 3510：删除 L1→GM、GM→L0A/L0B 直连；
新增 L0C→UBUF 与 UBUF→L1 两条单向通路

2 Ascend 新增 alloc / copy 接口

tilelang/ascend/language —— 相对通用 CUDA 面的差异

alloc 家族：按硬件层级直接分配

T.alloc_shared(shape, dtype)	UB（Unified Buffer）
T.alloc_l1(shape, dtype)	L1 Buffer（__cbuf__）
T.alloc_l0a(shape, dtype)	L0A（__ca__）
T.alloc_l0b(shape, dtype)	L0B（__cb__）
T.alloc_l0c(shape, dtype, layout=True)	L0C（__cc__），固定 layout
T.alloc_l0a_sf(buf, sf_dtype="uint16")	MX scale 句柄（零存储）
T.alloc_l0b_sf(buf, sf_dtype="uint16")	同上，L0B 侧

T.copy 新增关键字参数

transpose=True	dn2nz 转置 N/D（非 nd2nz）
l2_cache_ctrl="NOTALLOC_KEEP"	L2 cache 策略（后缀匹配）
pad_value=v	行宽补到 32B 边界并填 v
data_select=True	复用硬件 pad 寄存器当前值
unit_flag_ctrl=	交给后续 MAD 的 unit flag
sub_blockid=	把这次搬运路由到某个 sub-block

新增 copy 变体

T.dual_copy(src, dst)	硬件双目标拷贝，M/N 拆给两个 AIV

适配要点

- alloc_shared 在 Ascend 上分配的是 UB，不是 CUDA 的 shared
- alloc_l0a / l0b 的 layout 延后到 T.gemm，由其 transpose 推断
- 整 buffer copy 要求元素数相等，shape 可不同（[K,M] → [M,K]）

CANN

TileLang Cube编程

1 T.gemm • 两级操作数入口

Cube（AIC）上的矩阵乘；操作数可以给 L1，也可以自己管到 L0

① L1 操作数 —— 融合路径（编译器插 L1→L0 staging）

```
T.copy(A[m, k*BK], A_l1)        # GM → L1
T.copy(B[n, k*BK], B_l1)
T.gemm(A_l1, B_l1, C_l0c, transpose_B=True, clear_accum=(k == 0))
```

② L0 操作数 —— 显式路径（自己控 LoadData 时机）

```
l0a = T.alloc_l0a((MAD_M, MAD_K), "float16")
l0b = T.alloc_l0b((MAD_N, MAD_K), "float16")
T.copy(A_l1, l0a)               # L1 → L0A
T.gemm(l0a, l0b, C_l0c, transpose_B=True)
```

参数与约束

transpose_A / transpose_B
L1 路径目前只支持 NT —— trans_A=False, trans_B=True

clear_accum
是否清零累加器（K 循环首轮置 True）

unit_flag_ctrl
Cube unit-flag，与后继的累加器 drain 配对

L0 region
指定实际参与的 MAD M/N/K

padding
分配与生产者 copy 可为对齐加 padding

2 T.gemm_blockscaled • MXFP8 量化矩阵乘

数据是 fp8，每 32 个 K 元素共用一个 E8M0 缩放因子；scale 存在 L0 tile 旁边的固定槽位里

① SF 句柄 —— L0 入口（scale 随 l0a 一起搬入）

```
l0a    = T.alloc_l0a((MAD_M, MAD_K), "float8_e4m3fn")
l0_sfa = T.alloc_l0a_sf(l0a, sf_dtype="int16")  # 绑到 l0a
T.copy(l1a,    l0a)                    # 数据 → L0A
T.copy(l1_sfa, l0_sfa)                 # scale 随之装上
T.gemm_blockscaled(l0a, l0b, l0c, l0_sfa, l0_sfb, transpose_B=True)
```

② SF buffer —— L1 入口（真存储）

```
l1_sfa = T.alloc_l1((BLOCK_M, sf_pairs), "int16")
T.gemm_blockscaled(l1a, l1b, l0c, l1_sfa, l1_sfb, transpose_B=True)
```

参数与约束

SFA / SFB
必需。L1 输入时是 L1 的 SF buffer；L0 输入时是 MX 槽位句柄

句柄绑定
必须为对应数据 buffer 分配，且选中相同的 leading index

k_start / granularity
隐式：从 SFA region slice 推 K offset，不用手动传

sf_dtype
8bit 或 16bit；uint16 = 一个元素装两个 E8M0，覆盖 64 个 K

槽位复用
槽位是粘的：一次 scale load 可服务多次 data load

SF 布局
make_ascend_sf_layout：[.., MN/16, K/(16/bits), 16, 16/bits]

CANN

TileLang Vector 编程

1 T.SimdVF • 寄存器级 MicroAPI

没有线程概念：直接对 Vector 寄存器下指令，谓词掩码自己管

```
with T.SimdVF():
    v = T.simd.alloc_local(4, "float32")     # 寄存器槽位组 ×4
    mask = T.simd.pset(32)                   # 谓词掩码
    for i in T.Unroll(4, explicit=True):
        v[i] = T.simd.vld(a[i * VL])         # 下标访问某个槽位
        v[i] = T.simd.vrelu(v[i], mask)      # 逐槽运算
    for i in T.Unroll(4, explicit=True):
        T.simd.vsts(c[i * VL], v[i], mask)   # 逐槽存回
```

· alloc_local 给出的是可寻址的寄存器槽位组，v[i] 直接下标访问

· alloc_var 是单寄存器变量，用返回值形式，不需要下标

· 掩码由 pset / pge 产生，多数算子按 (a, b, mask) 三参调用

寄存器槽位组申请

T.simd.alloc_local(shape, dtype)
可寻址寄存器数组（scope local），用 v[i] 逐槽访问

T.simd.alloc_var(dtype)
单个寄存器变量（scope local.var），返回式风格

T.simd.* 算子分类（约 80 个）

访存 / 搬运	vld vld2 vsts vsstb vgatherb vscatter pld pst
计算类	vadd vsub vmul vdiv vmax vmin vmadd vand vshl vrelu
归约 / 排列	vcadd vcgadd vcmax vcgmax vintlv vdintlv vpack vunpack vselr
谓词 / 比较	pset pge vcmp vcmps pand por pxor pnot psel ppack
类型转换 / 生成	vcvt（带 precision 选择器）· vdup vdupv vci · chistv2

2 T.SimtVF • 线程模型

threads 默认 128，最多 3 个线程维度；T.Parallel 把迭代映射到线程

```
with T.SimtVF(threads=128):
    for i, j in T.Parallel(block_M // 2, block_N):
        C_ub[i, j] = T.max(C_ub[i, j], 0)    # epilogue 逐元素
```

· 最典型的用法：epilogue 里的逐元素操作，一行 T.Parallel 搞定

· coalesced_width（T.Parallel / T.copy 通用）是每线程向量化宽度上限，撑不住则降级并告警

T.SimtVF 的主要应用场景

随机数生成	必须：T.rng_init 只允许在 SimtVF 内
离散访存	UB、GM 的离散访存和原子操作
需屏障的并行循环	仅此：编译器在 SIMT_VF 内自动插 asc_syncthreads()
跨线程归约	常规：reducer 的 epoch 模型（RMSNorm、decode softmax）

代价：kernel 内只要有一个 SimtVF 区，UB 就从 248 KB 降到 216 KB（扣 32 KB 线程上下文）

TileLang的流水与调度原语介绍

1 T.Pipelined • C/V 流水可各自成环，亦可合并成环

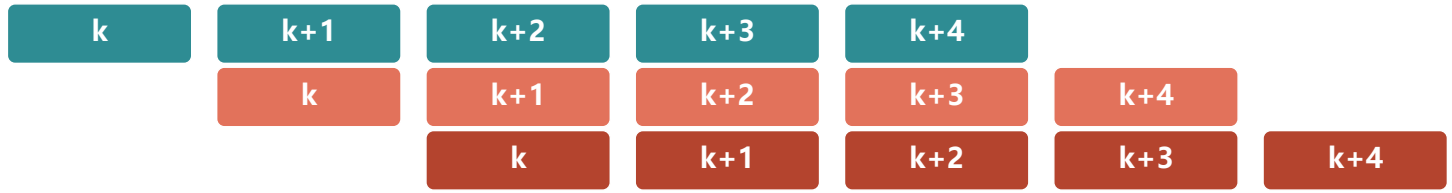
版本数由 Z3 按各 scope 的容量预算分配；每一段可独立设 stage 数

AIC / Cube 侧 —— 矩阵流水　MTE2 → MTE1 → Cube

MTE2
GM → L1 · num_l1_stages

MTE1
L1 → L0A/L0B · num_l0_stages

Cube
MAD（矩阵计算，跑在 AIC 上）



AIV / Vector 侧 —— 向量流水　MTE2 → Vector → MTE3

MTE2
GM → UB · annotate_buffer_versions

Vector
UB 上的 SIMT / SIMD epilogue

MTE3
UB → GM



Cube 与 Vector 可连进同一个 T.Kernel：L0C → Fixpipe → UB → Vector　同一 pipe 每 II 拍发一个 task；不同 pipe 之间自由重叠

2 T.Persistent

确定 block num，返回当前 block 处理的 tile id

```
for tile in T.Persistent([M // BM * n_tiles], num_blocks, bx):
    m, n = tile // n_tiles * BM, tile % n_tiles * BN
    for k in T.Pipelined(K // BK, num_stages=2):
        T.copy(A[m, k*BK], A_l1)
        T.gemm(A_l1, B_l1, C_l0c, transpose_B=True, clear_accum=(k==0))
    T.dual_copy(C_l0c, C_ub)
```

与流水的关系：两种写法

情况一 • 直接由 num_stages 控制
T.Persistent([...], num_blocks, bx, num_stages=N)

情况二 • 结合 T.Pipelined，单独控制内部流水
T.Pipelined(K // BK, num_stages=M)　　# 写在 tile 循环内

3 T.Parallel

SIMT / SIMD 并行处理

T.SimtVF(threads=128)

SIMT 区 · 最多 3 个线程维度

T.Parallel 在线程间分发，编译器插屏障

T.SimdVF()

SIMD 区 · 不带线程数

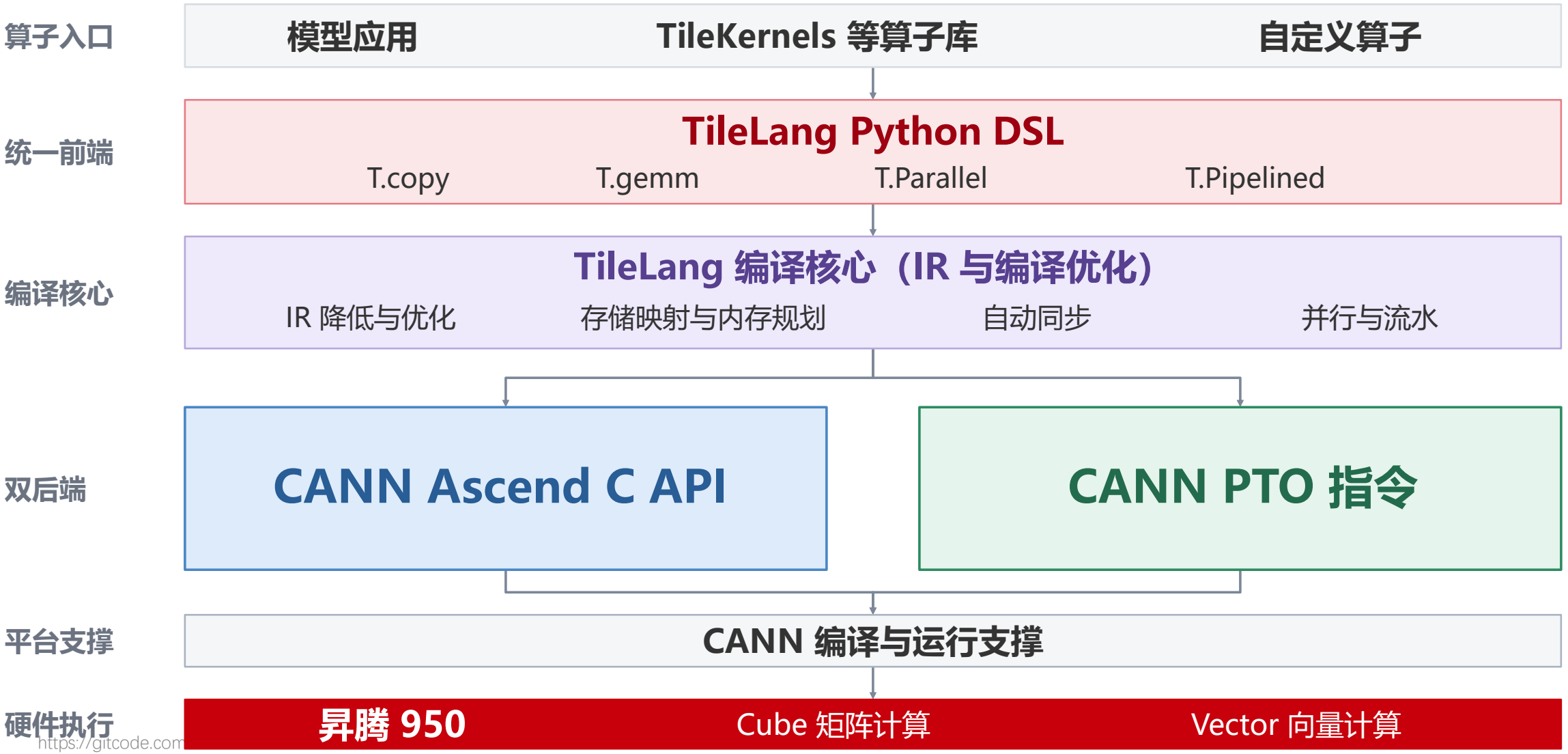
降到 MicroAPI 寄存器级向量指令

VF 块的硬约束（VFChecker 强制）

· 所有 T.Parallel 必须在 T.SimdVF() 或 T.SimtVF() 内
· VF 块外的 T.copy 必须 src/dst dtype 一致（DMA 不做类型转换，要 cast 进 VF）
· SIMD_VF 不能访问 global memory
· VF 块内分配的 buffer 不能逃出该 VF scope
· VF 外分配的 local.var 不能在 VF 内修改

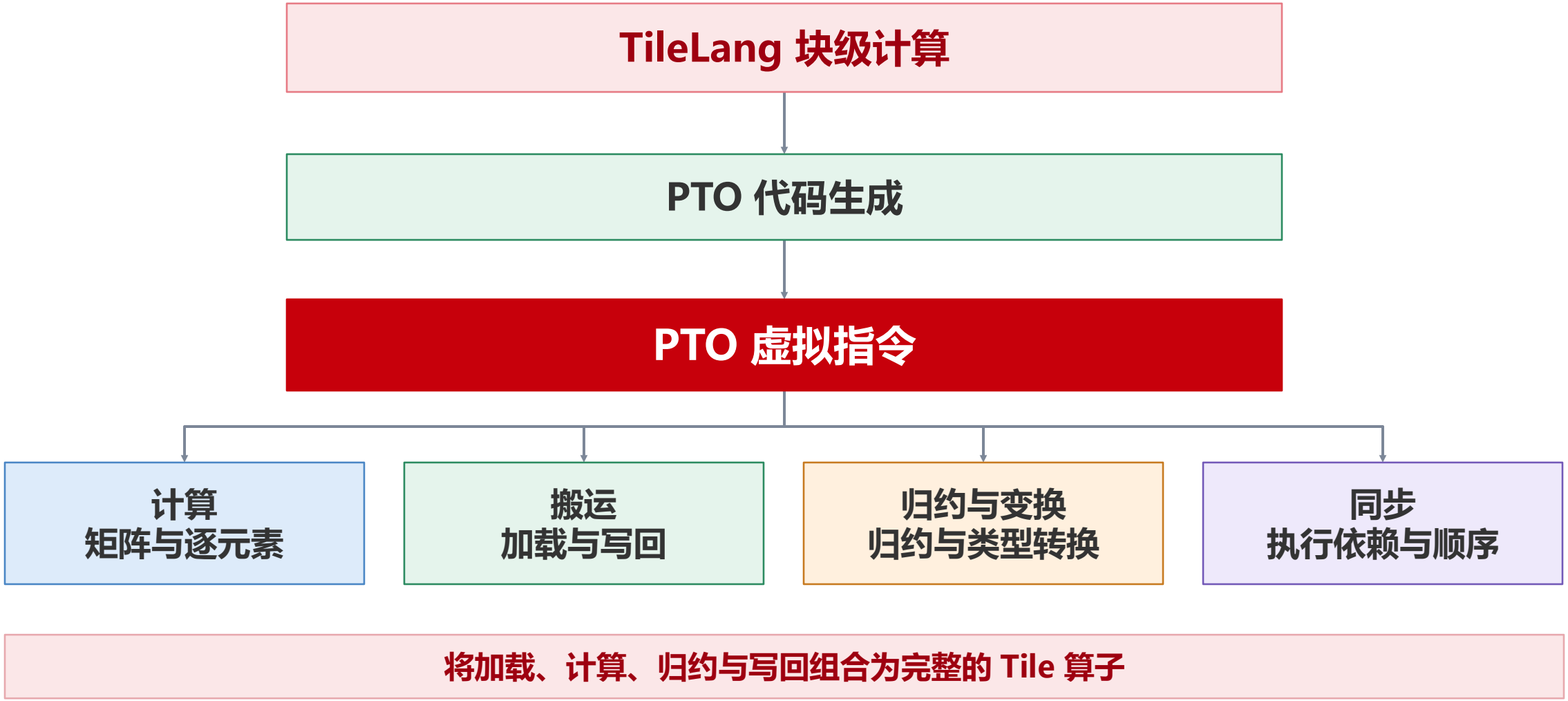
CANN

TileLang软件架构



https://gitcode.com

PTO新特性支持


TileLang 块级计算
PTO 代码生成
PTO 虚拟指令
计算
矩阵与逐元素
搬运
加载与写回
归约与变换
归约与类型转换
同步
执行依赖与顺序
将加载、计算、归约与写回组合为完整的 Tile 算子

PTO虚拟指令集提出的背景：跨代兼容

挑战：PTO 兼容新旧核架构， 需要“兼容不降速”

- 基于已有PTO指令接口，做到微架构高度适配
- 增强PTO指令约束，满足新架构PPA要求

挑战：算法变动快，PTO需要兼顾易用性和高性能

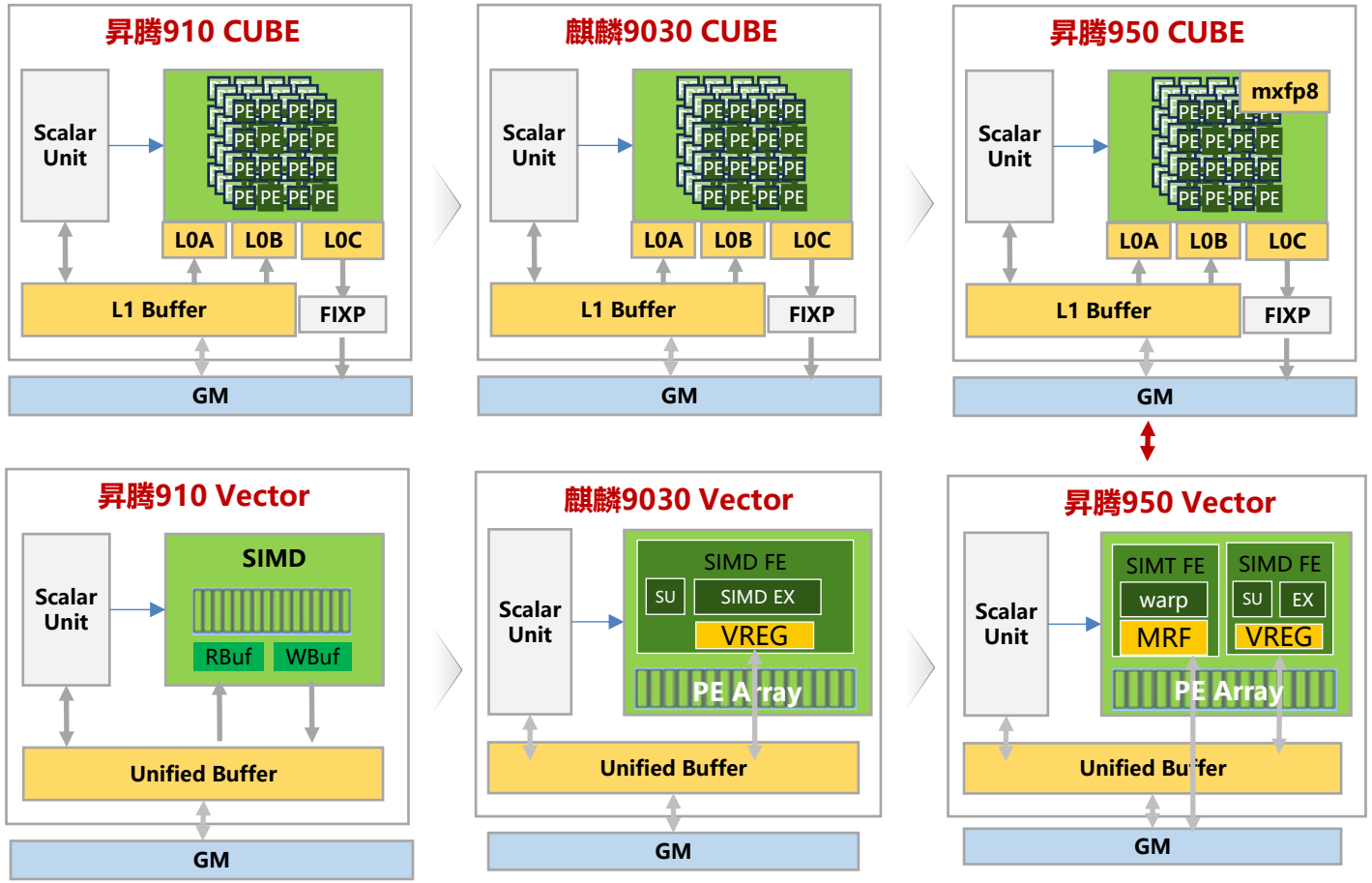
- 提供更方便的排流水的优化
- Profiling提供最够信息更方便融合和软件优化

挑战：软硬协同深水区，PTO要封装DSA定制

- 在PTO的约束下，进行有限度DSA硬化
- 承诺不改变访存层级，不改变Memory Model

挑战：开放接口，吸引生态使用

- 主动对接TileLang，PyPTO等新语言
- 持续改进，提升易用性



CANN

PTO虚拟指令集：如下平台已经支持





1月30日
PTO指令集支持麒麟9030



昇腾 384 超节点



3月30日
PTO指令集支持昇腾950芯片



6月30日
PTO指令集支持车载芯片

CANN

PTO虚拟指令集：目前支持的语言

PyPTO

MPMD调度的Tensor编程框架

TileLang
TILE LANGUAGE

SPMD调度的Tile编程框架

PTO

跨平台
PTO文件
字节码

PTOAS







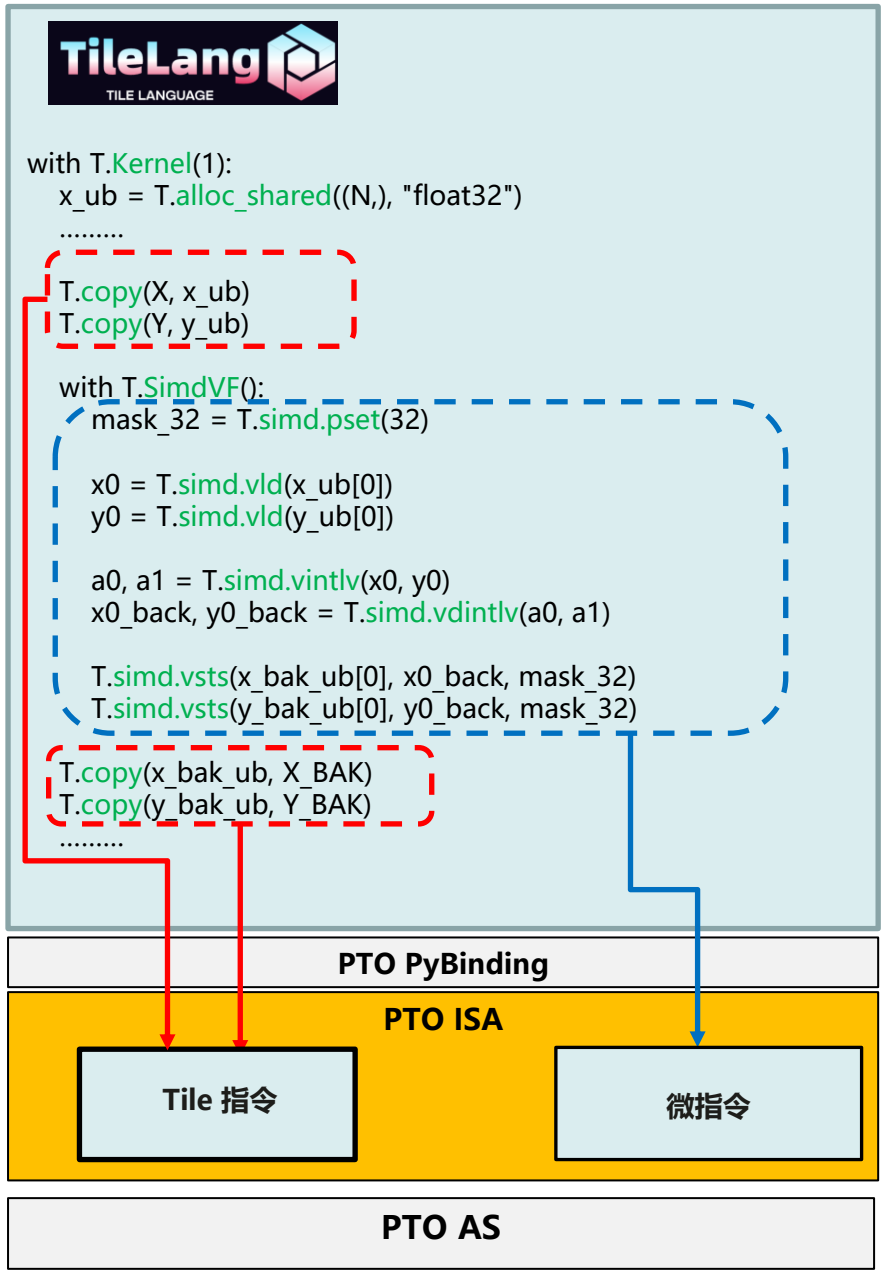
昇腾 384 超节点





CANN

930开源TileLang对接PTO的方式



Tile接口与simd向量接口混合编程

为Expert提供手写simd VF能力

TileLang负责流水排布，提供极致性能

提供底层细粒度的编程能力

在前端直接表达流水排布和Layout

直接调用底层simd指令接口获取极致性能

Tile指令由微指令搭建而成

TileLang使用自定义的T.COPY()

PyPTO使用PTO TileOP

跨代际兼容

后续NPU芯片软件后向兼容(仅限Tile指令)

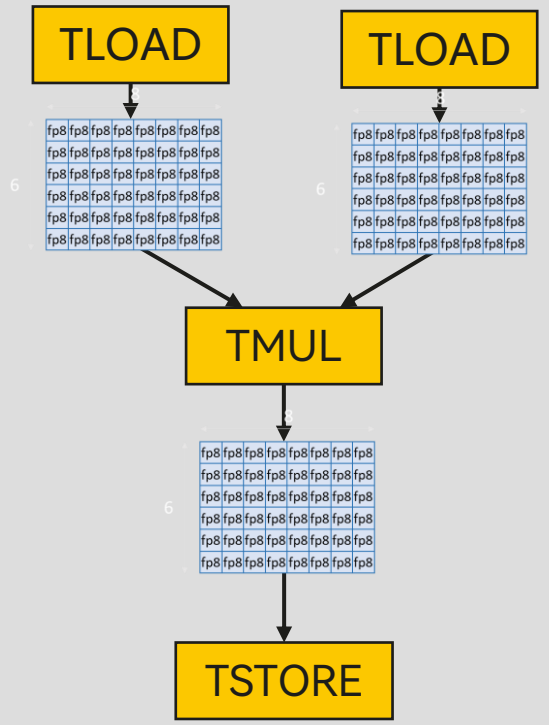
CANN

PTO-ISA VMI扩展介绍

Tile 指令集

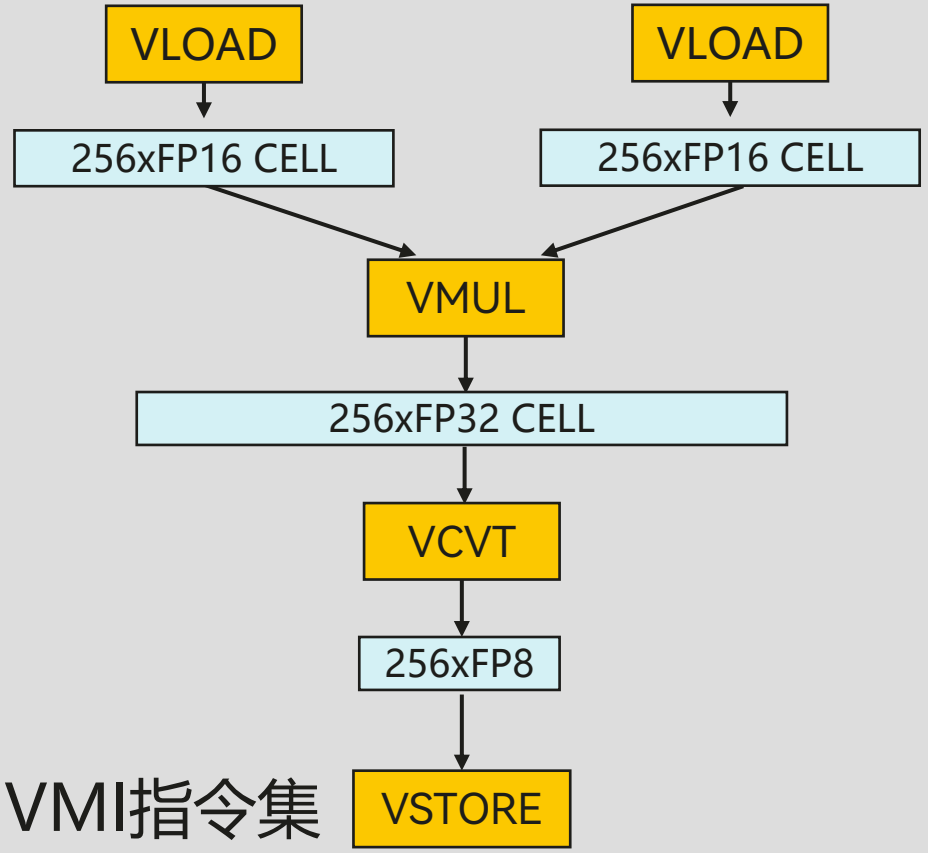
操作码	TILE	TILE

```
pto.tload %tensorA -> %ta
pto.tload %tensorB -> %tb
pto.tmul %ta, %tb -> %tc
pto.tstore %tc, %tensorC
```



操作码	CELL	CELL

```
pto.vmi.vload %tensorA -> %ta
pto.vmi.vload %tensorB -> %tb
pto.vmi.vmul %ta, %tb -> %tc
pto.vmi.vstore %tc, %tensorC
```



VMI指令集

CELL就是一维Tile

一维Tile和多维Tile接口一致

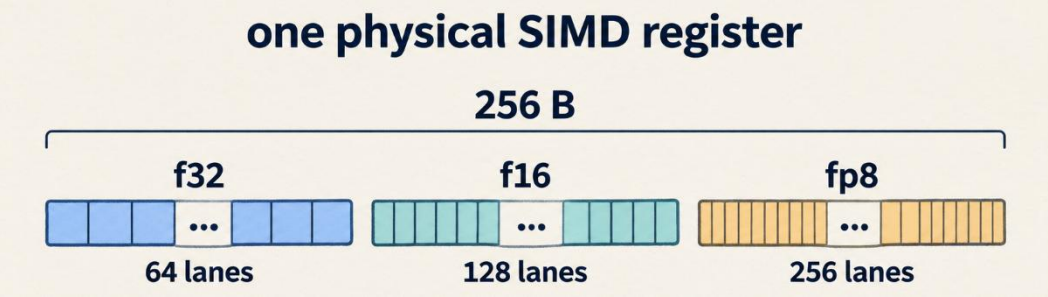
无需考虑向量宽度，提供极致性能

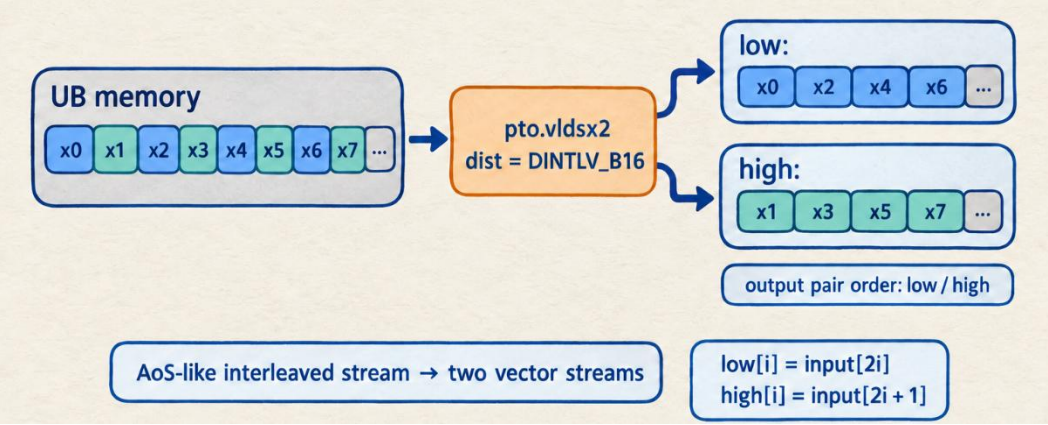
PTOAS做寄存器layout推导

PTO指令集全量后向兼容

CANN

TileLang SIMD编程复杂度的由来






```
with T.SimdVF():
    # -- 加载 reciprocal scale:128 个 f16,dist = E2B_B16(循环不变,提到循环外)--
    scale_128 = T.simd.vld(scale_ub[0], dist='E2B_B16')        # float16x128
    scale_fp32 = T.simd.vcvt(scale_128, T.float32, part=0)     # EVEN → float32x64

    for i in T.serial(block_count):
        offset = i * load_stride

        # (1) 加载 256 个 f16 → DINTLV_B16 交错拆成 low(128) + high(128)
        low, high = T.simd.vld2(x_ub[offset], dist='DINTLV_B16')

        # (2) f16 → f32:low/high 各拆 EVEN/ODD → 4 个 vreg<64xf32>
        cvt_low_even  = T.simd.vcvt(low,  T.float32, part=0)       # EVEN
        cvt_high_even = T.simd.vcvt(high, T.float32, part=0)
        cvt_low_odd   = T.simd.vcvt(low,  T.float32, part=1)       # ODD
        cvt_high_odd  = T.simd.vcvt(high, T.float32, part=1)

        # (3) 乘 reciprocal scale:4 条独立 vmul(同一 scale 广播到 4 路)
        mul0 = T.simd.vmul(cvt_low_even,  scale_fp32)
        mul1 = T.simd.vmul(cvt_high_even, scale_fp32)
        mul2 = T.simd.vmul(cvt_low_odd,   scale_fp32)
        mul3 = T.simd.vmul(cvt_high_odd,  scale_fp32)

        # (4) f32 → fp8:4 条 vcvt,分别打入 P0/P1/P2/P3 part
        p0 = T.simd.vcvt(mul0, T.float8_e4m3fn, part=0)          # P0
        p1 = T.simd.vcvt(mul1, T.float8_e4m3fn, part=1)          # P1
        p2 = T.simd.vcvt(mul2, T.float8_e4m3fn, part=2)          # P2
        p3 = T.simd.vcvt(mul3, T.float8_e4m3fn, part=3)          # P3

        # (5) 合并:3 条 vor 把 P0~P3 拼回一个完整 256 元素向量
        merge01  = T.simd.vor(T.reinterpret(p0, 'uint8x256'), T.reinterpret(p1, 'uint8x256'))
        merge012 = T.simd.vor(merge01, T.reinterpret(p2, 'uint8x256'))
        merged   = T.simd.vor(merge012, T.reinterpret(p3, 'uint8x256'))

        # (6) 写回
        T.simd.vsts(out_ub[offset], T.reinterpret(merged, 'float8_e4m3fnx256'),
dist='NORM_B8')
```

后续TileLang对接PTO VMI将简化用户的编程复杂度

w/o PTO vmi

```
with T.SimdVF():
    # -- 加载 reciprocal scale:128 个 f16,dist = E2B_B16(循环不变,提到循环外)--
    scale_128 = T.simd.vld(scale_ub[0], dist='E2B_B16')        # float16x128
    scale_fp32 = T.simd.vcvt(scale_128, T.float32, part=0)     # EVEN → float32x64

    for i in T.serial(block_count):
        offset = i * load_stride

        # (1) 加载 256 个 f16 → DINTLV_B16 交错拆成 low(128) + high(128)
        low, high = T.simd.vld2(x_ub[offset], dist='DINTLV_B16')

        # (2) f16 → f32:low/high 各拆 EVEN/ODD → 4 个 vreg<64xf32>
        cvt_low_even  = T.simd.vcvt(low,  T.float32, part=0)       # EVEN
        cvt_high_even = T.simd.vcvt(high, T.float32, part=0)
        cvt_low_odd   = T.simd.vcvt(low,  T.float32, part=1)       # ODD
        cvt_high_odd  = T.simd.vcvt(high, T.float32, part=1)

        # (3) 乘 reciprocal scale:4 条独立 vmul(同一 scale 广播到 4 路)
        mul0 = T.simd.vmul(cvt_low_even,  scale_fp32)
        mul1 = T.simd.vmul(cvt_high_even, scale_fp32)
        mul2 = T.simd.vmul(cvt_low_odd,   scale_fp32)
        mul3 = T.simd.vmul(cvt_high_odd,  scale_fp32)

        # (4) f32 → fp8:4 条 vcvt,分别打入 P0/P1/P2/P3 part
        p0 = T.simd.vcvt(mul0, T.float8_e4m3fn, part=0)          # P0
        p1 = T.simd.vcvt(mul1, T.float8_e4m3fn, part=1)          # P1
        p2 = T.simd.vcvt(mul2, T.float8_e4m3fn, part=2)          # P2
        p3 = T.simd.vcvt(mul3, T.float8_e4m3fn, part=3)          # P3

        # (5) 合并:3 条 vor 把 P0~P3 拼回一个完整 256 元素向量
        merge01  = T.simd.vor(T.reinterpret(p0, 'uint8x256'), T.reinterpret(p1, 'uint8x256'))
        merge012 = T.simd.vor(merge01, T.reinterpret(p2, 'uint8x256'))
        merged   = T.simd.vor(merge012, T.reinterpret(p3, 'uint8x256'))

        # (6) 写回
        T.simd.vsts(out_ub[offset], T.reinterpret(merged, 'float8_e4m3fnx256'),
dist='NORM_B8')
```

with PTO vmi

```
with T.SimdVF():
    # -- 加载 reciprocal scale:8 个 f16 → vbrc{group=8} 广播到 256 → cvt f32(循环不变,提到循环
外)--
    scale_f16 = T.vmi.vload(recip_ub[0, 0], size=GROUPS_PER_STRIP)
    scale_f16_vec = T.vmi.vbrc(scale_f16, size=VL, group=GROUPS_PER_STRIP)
    scale_fp32 = T.vmi.vcvt(scale_f16_vec, 'float32')

    for i in range(block_m):
        for strip in T.serial(n_strips):
            offset = strip * VL
            # (1) 加载 256 个 f16(无需 DINTLV,编译器自动拆 low/high)
            x_f16 = T.vmi.vload(x_ub[i, offset], size=VL)

            # (2) f16 → f32(无需 EVEN/ODD,一条 vcvt 出整条 256)
            x_fp32 = T.vmi.vcvt(x_f16, 'float32')



            # (3) 乘 reciprocal scale(一条 vmul,scale 已在寄存器内广播到 256)
            res_fp32 = T.vmi.vmul(x_fp32, scale_fp32)



            # (4) f32 → fp8(无需 P0~P3 part,一条 sat vcvt 出整条 256)
            res_fp8 = T.vmi.vcvt(res_fp32, 'float8_e4m3fn', rounding='R', saturate='SAT')

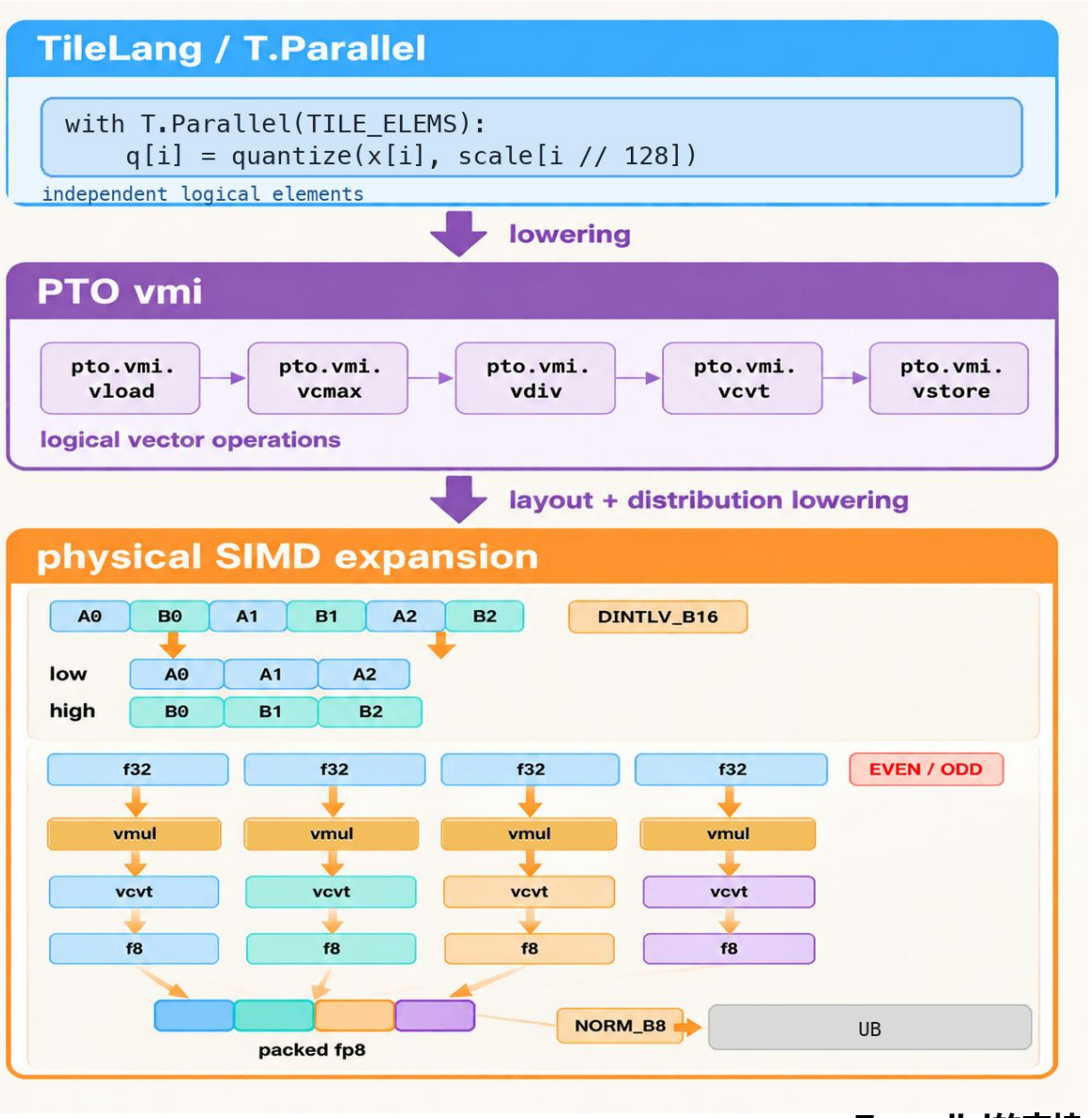


            # (5) 合并 — 无需 vor,步骤 (4) 已直接给出完整 256 元素向量



            # (6) 写回
            T.vmi.vstore(res_fp8, y_ub[i, offset])
```

PTO后续规划



T.parallel的支持

多平台开源

GitCode pto-isa
gitcode.com/cann/pto-isa

GitHub pto-as
github.com/PTO-ISA/PTOAS

GitHub TileLang
github.com/PTO-ISA/tilelang

GitHub kernels
github.com/PTO-ISA/tilelang-puzzles-ascend

CANN

TileKernels 的昇腾 950 算子支持

mHC（9）

Sinkhorn 矩阵归一化
Pre / Post 混合与拆分
多层重计算融合

量化（12）

FP8 / FP4 量化与反量化
逐 token、分块、逐通道
SwiGLU / RMSNorm 融合

MoE（4）

Top-k 专家选择
路由打分与权重归一化
前向与反向计算

Engram（6）

门控前后向、权重梯度
权重融合、Hash、Sinkhorn

RoPE（1）

原地旋转位置编码
rot_dim 支持 64 / 128

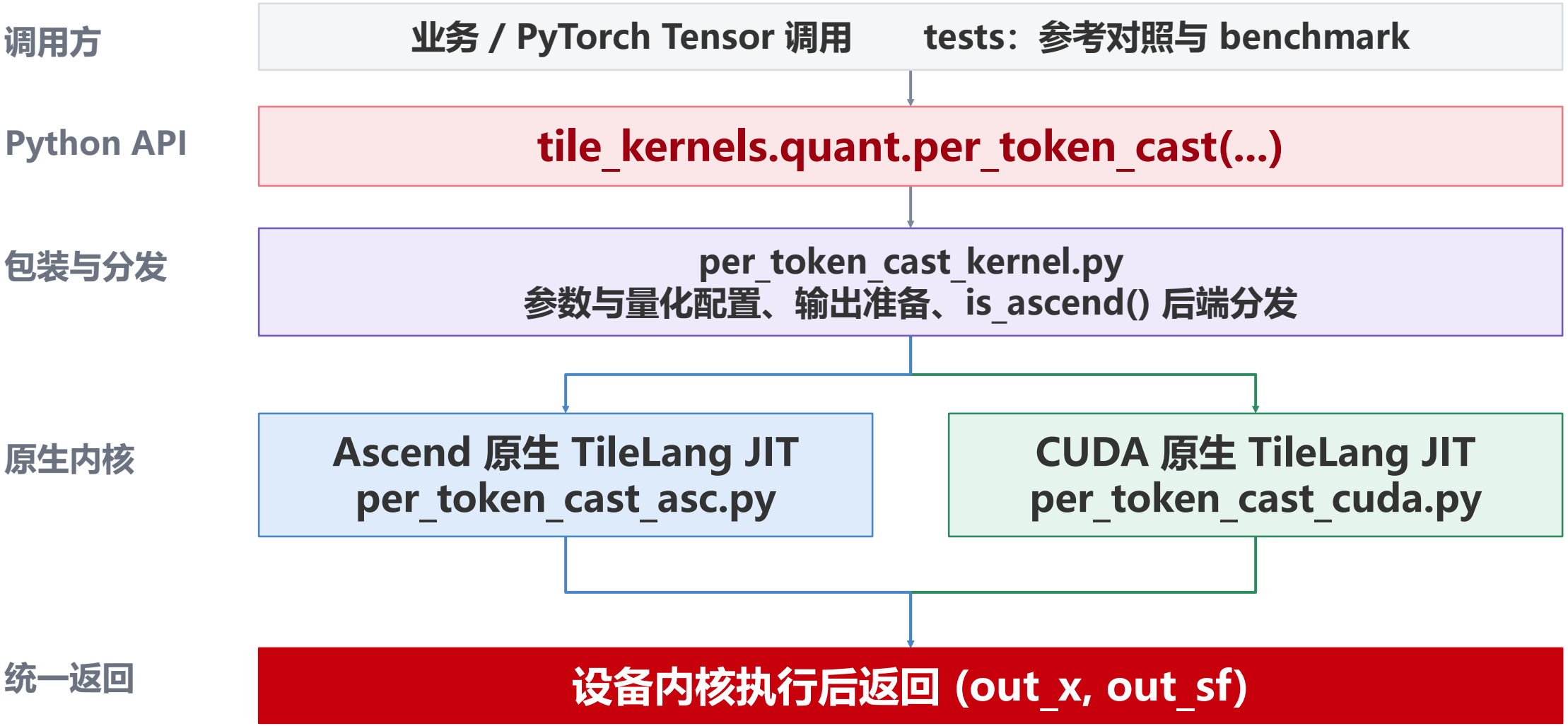
Rand（1）

正态随机数生成
BF16 / FP32；set_seed

统一 Python API：运行环境自动选择 CUDA 或 Ascend 后端

CANN

TileKernels 的调用与后端架构



Tile Kernel 编程：分块、内存与搬运

分块加法的核心代码

```
N = 65536
TILE_ELEMS = 1024
OUT_TILES = N // TILE_ELEMS

with T.Kernel(OUT_TILES) as tile:
    A_ub = T.alloc_shared((TILE_ELEMS,), "float32")
    B_ub = T.alloc_shared((TILE_ELEMS,), "float32")
    C_ub = T.alloc_shared((TILE_ELEMS,), "float32")

    start = tile * TILE_ELEMS
    end = start + TILE_ELEMS
    T.copy(A[start:end], A_ub)
    T.copy(B[start:end], B_ub)
    with T.SimdVF():
        for i in T.Parallel(TILE_ELEMS):
            C_ub[i] = A_ub[i] + B_ub[i]
    T.copy(C_ub, C[start:end])
```

分块：每次处理 1,024 个元素

65,536 个元素划成 64 个 Tile
tile=3 对应区间 [3,072 : 4,096)

alloc：声明容量与存储层级

alloc_shared 在 Ascend 上对应 UB
每个 FP32 缓冲区占 4 KB

```
X_l1 = T.alloc_l1((BM, BK), "float16")
C_l0c = T.alloc_l0c((BM, BN), "float32")
```

矩阵算子用 L1 放输入块，L0C 放累加结果

copy：显式写出读写关系

输入块从 GM 搬入 UB
结果块从 UB 写回 GM
编译器按源、目的存储层级选择搬运路径

前端指定 Tile 大小、存储层级和读写关系，编译器规划缓冲地址并生成搬运指令

mHC pre_big_fuse：融合算子编程示例

GEMM 结果 → RMSNorm 后处理 / mix 拆分 → Sinkhorn → residual apply mix

GEMM 先计算，后续阶段合并到 pre_big_fuse Kernel；输出 post_mix、comb_mix 和 layer_input

配套 reduction / split 阶段

```
for tile_id in T.Persistent(
    [T.ceildiv(num_tokens, token_block_size)],
    n_cores, core_id, group_size=1, num_stages=2):
    ...
with T.SimtVF(threads=_VEC):
    for lane in T.Parallel(_VEC):
        ...
```

Persistent 分配 token Tile；Parallel 在线程域内分工。
num_stages=2 为这段遍历声明两级流水。

融合 Kernel：SIMT 整理，SIMD 计算

```
with T.SimtVF(threads=1024):
    load_thread = T.get_thread_binding()
    mixes_ub[mix_index, token_lane] = mix_sum

with T.SimdVF():
    mix_pre = S.vmul(S.vld(mixes_ub[j, 0]), rms)
    ...  # sigmoid / Sinkhorn / apply mix
```

SimtVF 用线程索引整理 token 与 mix 布局。
SimdVF 用向量寄存器处理归一化、门控和混合。

融合 Kernel：沿 hidden Tile 组织搬运与计算流水

```
for hidden_tile in T.Pipelined(
    num_hidden_tiles, num_stages=residual_num_stages):
    T.copy(residual[..., hidden_start:hidden_start+h_blk], residual_ub)
    with T.SimdVF(): ...  # 使用 pre_mix_ub 计算 layer_input_ub
    T.copy(layer_input_ub, layer_input[..., hidden_start:hidden_start+h_blk])
```

alloc_shared 声明 UB Tile
copy 表达 residual 读入和结果写回
Pipelined 暴露搬运与计算重叠
pre_mix 在片上接续 apply mix

融合保留在一个 Kernel 内的片上数据流，SIMT / SIMD 作用域分别表达线程与向量执行

CANN

TileLang算子开发新模式

开发课程

cann-learning-hub

TileLang 算子开发课程
编程基础、精度验证与优化

课程入门

https://gitcode.com/cann/cann-learning-hub/tree/test/tutorials/tilelang_operator_development

算子库

ops-tilelang

面向昇腾的算子实现
参考、复用与持续优化

算子实践

https://gitcode.com/cann/ops-tilelang

Agent 辅助

cannbot-tilelang

辅助算子生成与调优
方案设计、代码生成与验证

Agent 辅助开发

https://gitcode.com/cann/cannbot-skills/tree/master/plugins-official/tilelang-op-orchestrator

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann